

Lessons Learned

Integration Traps from the Project

Gurmann Basran
Revised 2026-06-15 (rev. 3).

1 Why this document exists

Each of the traps in this document cost me real debugging time, somewhere between one and eight hours each, and in one case an evening where I nearly lost internet for my whole apartment. None of them were findable by reading the official documentation of the tools involved ahead of time. Every one of them generalizes beyond my specific project, which is why I have written each one at the *class of trap* level. Specific tool names, config paths, and version strings have been kept out because they are attacker-useful fingerprints; what is preserved is the pattern itself.

The point of keeping a document like this is so that future-me does not re-discover the same lesson on a Tuesday at 2 AM. The secondary point is that if you are reading this and you work with anything in the same category of tool, one of these might save you an afternoon. This revision adds the traps from the operational-health work, the recovery-insurance milestone, and a surface-reduction teardown to the ones from the original build, the second hardening pass, and the workstation migration.

2 Do not re-serialize a config file when the tool treats it as a byte-exact artifact

Class of trap. Some tools treat their configuration file as a byte-exact artifact: they hash it, diff it, version-control it internally, or otherwise care about exact byte equality beyond XML semantics. A “just parse and re-serialize” library, which is a standard instinct for programmatic config manipulation, will reorder attributes, normalize whitespace, or change self-closing-tag formatting in ways that the tool’s internal diff interprets as “entire file changed.” The symptom is either a tool that refuses to render the changed config, or, worse, a tool that applies a merged version where half of the intended change has been silently dropped.

The right path. Always check whether the tool has a programmatic API before you write to its config file. Most modern appliances do; most of them are poorly documented but stable. Use the API and skip the file-level manipulation entirely.

Why this generalizes. Any system that treats its config as a byte-exact artifact cannot safely be edited by a library that does not preserve exact bytes. This applies to more tools than you would expect, especially in the network-appliance and storage-appliance category.

3 Some appliances silently ignore standard config drop-ins

Class of trap. The modern Linux convention for configuring services is a directory of drop-in files: you put your change in `/etc/<service>/conf.d/99-mine.conf` and the service reads it. This works on a stock distribution. It does not work on some appliances, where the on-disk config is regenerated from an internal declarative source and the drop-in directory simply is not referenced. The trap is that the file is there, the syntax is valid, the usual validation command parses it, but the service itself is not reading it. The hardening you thought you applied is silently inactive.

The right path. Check the config precedence order before assuming a drop-in works. On an appliance, the declarative source is usually the appliance's own config store, not the OS-level file layout. Edit the declarative source; let the appliance regenerate from it.

Why this generalizes. When an appliance generates its own config from a declarative source, OS-level best-practice layouts become misleading. Always find the generating source before you try to add to the generated file.

4 Some config options apply at resource-creation, not at service-restart

Class of trap. The mental model “change daemon config, restart daemon, change takes effect” is not always accurate. Some config options are evaluated at resource-creation time and baked into the resource's own configuration, then never re-read from the daemon config. Restart the daemon all you want; the existing resources will continue using the old settings, and only newly-created resources will pick up the change.

The right path. Check whether a config option is read at startup (retroactive to existing state) or at resource creation (not retroactive). For not-retroactive options, either re-create every existing resource or add a workaround that operates at a layer below the daemon.

Why this generalizes. “Restart takes effect” and “restart applies to existing state” are very different guarantees. This trap shows up in container runtimes, in logging daemons, in message brokers, anywhere that tracks per-resource state across restarts.

5 GUI panels labelled “custom configuration” are scoped

Class of trap. Web UIs often expose a panel labelled something like “Custom Configuration” for the underlying tool. The abstraction leaks: the panel is scoped to a specific context (a per-host block, a per-server block, a per-tenant block), and a directive that does not belong in that context will be accepted by the UI, saved to the regenerated config, and then cause the underlying daemon to refuse to start. The result is that a change you thought was additive takes down every service sharing that regenerated config.

The right path. Find and read the generated config before writing to a “Custom Configuration”

panel. Know which scope the panel's content ends up in. Directives that belong at a different scope need a different file, often not exposed through the UI at all.

Why this generalizes. Any configuration-management GUI that hides scope is a source of this class of bug. When the backing format has strict scoping rules, the abstraction will eventually leak.

6 OS-level library upgrades can silently break language bindings

Class of trap. Routine package upgrades on a services host can pull in a new version of a system library that has removed a symbol a language-level binding was relying on. The binding is still installed; the version number has not changed; but calling into it fails at import. A tool that has been running for months starts failing at startup with a cryptic error, and you are debugging a system library from inside a script.

The right path. Update the language-level binding to a version that matches the new system library. Add a startup healthcheck to tools that depend on C-level bindings so that an import failure produces a clear message rather than a cryptic one.

Why this generalizes. The “it was working yesterday” heuristic is unreliable on a system where the OS keeps itself up to date. Every critical third-party binding on a services host deserves a boot-time healthcheck.

7 Soft-delete without on-disk cleanup leaves orphans that block regeneration

Class of trap. Many tools that store configuration in a database plus filesystem treat a delete as a soft-delete: the row is flagged as deleted, but the corresponding on-disk artifact (a config file, a generated template, a cached build) is not removed. The trap surfaces weeks later when you try to create a new resource that shares a unique property with the deleted one (a hostname, a path, a route), and the new resource cannot be created because of an orphan artifact that still exists on disk.

The right path. After any delete that should have freed up a name, check the filesystem for orphans. When the tool offers a hard-delete, use it. When it does not, script the cleanup.

Why this generalizes. Any tool that treats its database and its filesystem as two separate sources of truth is susceptible to this. Soft-delete is a great feature until it is a silent source of conflict.

8 Legacy fallback paths survive credential rotation

Class of trap. Some daemons check multiple locations for credentials, keys, or authorized entities by default. You update the primary location, remove the old credential, confirm it no longer appears

in that location, and then the old credential continues to work anyway, because the daemon is checking a secondary location you did not know about.

The right path. When rotating credentials, audit *every* location the daemon will read them from. Either clean all locations, or explicitly configure the daemon to look at only one.

Why this generalizes. Legacy fallback paths are a class of “I thought I removed that access but actually I did not” bugs. They are especially common in tools that predate the modern single-canonical-location convention.

9 Command-line option compatibility is rarely documented completely

Class of trap. A command-line tool accepts several combinations of flags, but not all combinations are valid. Some flags only function with particular other flags; when combined with an incompatible alternative, they are silently ignored or cause a cryptic error. The documentation rarely enumerates every compatibility rule.

The right path. When a command takes an unusual combination of flags, test with a known-good simple case first, then add flags one at a time. When a flag you expected to do something appears to do nothing, check for an implicit dependency on another flag.

Why this generalizes. Command-line option matrices are almost never documented exhaustively. The only reliable way to know whether a combination works is to try it and observe.

10 A privilege-elevation helper can be gated by a broader policy than the rule you wrote

Class of trap. You write a tightly-scoped authorization rule: this user may run exactly this one action on exactly this one object, and nothing else. Then you reach for a convenience wrapper to actually run the privileged operation, and the wrapper authenticates against a different, generic policy action, the broad “run anything as administrator” gate, instead of the narrow rule you wrote. Your scoped rule becomes dead code and the operation is granted the full privilege you were trying to avoid. It passes a casual review because the narrow rule is sitting right there in the config, looking correct and doing nothing.

The right path. Drive the privileged operation through the channel that is actually evaluated against your fine-grained rule, install the rule where the policy engine truly reads it, and then write hostile negative tests that prove a wrong action and a wrong object are both denied. Authorization you have not watched fail is authorization you have not confirmed.

Why this generalizes. Access is only ever as tight as the policy action the runtime actually evaluates, which is frequently not the one your rule names. Any time you use a convenience elevation wrapper, verify which policy gate it trips, and prove the deny cases rather than trusting that a narrow-looking rule is the enforced path.

11 Hardening one layer can silently break another; test the whole boot path

Class of trap. You harden one subsystem for integrity, for example making an identity or resolver file immutable so that nothing can tamper with it, and in doing so you quietly remove a degree of freedom that a different subsystem depended on, because that other subsystem legitimately needed to write the file you just locked. Nothing breaks at the moment you make the change. It breaks on the next reboot, when an automatic address-assignment client cannot write what it needs, the interface comes up with no address, and the host is unreachable for a reason that has nothing obvious to do with the hardening you applied a week earlier.

The right path. When you lock a resource that another component writes, remove that component's need to write it rather than just denying it. Then exercise the entire cold-boot path end to end, not the steady state, because the failure mode you introduced hides until the next restart.

Why this generalizes. Hardening changes have non-local effects; a control that improves one layer's integrity can take away something another layer was quietly relying on. Any immutability or lockdown change earns a full restart test before you call it done.

12 Choose a foundational tool by comparison, not by default

Class of trap. You pick a foundational, expensive-to-replace component, the kind everything else gets built on top of, because it is the popular default and it looks good in a screenshot, without comparing it against the handful of real alternatives on the dimensions that will actually matter for your use. Months and a great deal of component-specific work later, you hit a capability it does not support or an area where it has been abandoned, and switching now means forfeiting most of that sunk work.

The right path. Before committing to any foundational stack, stop and lay out the top three to five alternatives with their explicit trade-offs, name the specific features you cannot live without and which alternatives support them natively, decide which of your priorities dominates, and choose on fit. If you cannot name three alternatives you considered and rejected, you have not researched the decision; you have defaulted into it.

Why this generalizes. The cost of a wrong foundational choice scales with how much you build on top of it, and the cheapest moment to evaluate it is before any of that weight exists. "It is the obvious popular pick" is precisely the rationalization that skips the comparison that would have surfaced the disqualifying gap.

13 Restart ordering can resurrect an address collision you thought was impossible

Class of trap. Some components in a system claim fixed addresses while others draw addresses dynamically from the same pool. As long as the startup order is stable, the fixed claims and the

dynamic draws never collide, and everything looks fine for months. Then a hard restart shuffles the startup order, a dynamically-assigned component grabs an address that a fixed-address component reserves, and whichever one starts first wins. The loser fails to come up with a misleading “address already in use” error that points at the wrong layer entirely.

The right path. Guarantee uniqueness across the whole address space rather than only among the fixed assignments: put the fixed addresses outside the dynamic allocation range, audit every assignment for collisions after any change that adds a member, and re-verify the live mapping after any hard restart.

Why this generalizes. Any system that mixes fixed and dynamic identifiers drawn from a shared pool has a latent collision that only appears under non-default startup ordering. Do not assume the startup order is stable; make uniqueness true by construction so the order cannot matter.

14 A host-only script escapes every audit scoped to the repository

Class of trap. You run a security audit scoped to your repository: you rewrite every script to pull secrets safely, you grep the tree for the forbidden patterns, you commit the fixes. But a script that was deployed straight to a host long ago and never copied back into version control is invisible to any search of the repository, so it sails through the audit untouched, still posting without authentication and still using the exact shortcut you just banned everywhere else, silently, for the entire window after the audit you believed was complete.

The right path. After every repository-scoped audit, enumerate the executable files on each host’s actual filesystem and diff them against the repository. Treat any host-only file as suspect: pull it back into version control, remediate it, redeploy it through the canonical pipeline, and commit. Enumerate by “is this executable,” not by file extension, because an extension-less script slips through a name-glob.

Why this generalizes. The repository is not the source of truth; each host’s filesystem is its own truth, and the two drift apart over time. Any audit that scopes only to the repository has a blind spot exactly the size of “things that live only on a host,” so the audit has to reach the host filesystem directly.

15 A security audit with no operational-health dimension is blind to the failures that take you down

Class of trap. A thorough security audit measures configuration correctness, access control, and attack surface. It does not measure whether a host has headroom, whether something is consuming a resource without bound, or whether the system will stay up under its own legitimate load. Those are temporal and capacity questions, and a point-in-time configuration audit cannot answer them, because the failure is not in any single snapshot; it is in a trend across hours. The trap is that the audit passes clean, every domain green, and then the system falls over to a runaway process or an

out-of-memory event that was never in scope. A perfect security posture and an operational outage coexist comfortably.

The right path. Treat operational health as its own audit dimension, separate from the security one, and give it a different mechanism. The security audit can be a point-in-time pass; the operational one cannot, because the property it checks is a trend. It has to run continuously: sustained processor saturation, memory and swap pressure, unbounded growth, filesystem fill, each watched on an interval rather than verified once. Tag its findings by the failure they expose, not by an attacker-facing severity, because the question has changed from “how bad is this misconfiguration” to “what are we blind to.”

Why this generalizes. Any system whose failure modes include resource exhaustion needs operational health monitoring that is independent of its security hardening, because the two ask different questions: the security audit asks “can an attacker reach this,” and the operational audit asks “will this stay up on its own.” A green security audit is not evidence of the second thing, and quietly treating it as if it were is how a well-secured system surprises you by falling over.

16 The monitor that watches a host should not live on that host

Class of trap. To save a machine, you run the monitoring and alerting stack on the same host as the services it watches. Under normal load this is fine. Under the exact conditions you most need an alert, memory pressure, an out-of-memory event, a full disk, the monitoring stack is a victim too. If the kernel’s out-of-memory killer reaps the alerting daemon instead of the noisy service, or the disk that fills is the one the alerter writes to, the host dies in silence and you find out hours later from the absence of a heartbeat. The detection inverts at the worst possible moment.

The right path. Put each safety-critical check on a host that does not depend on the thing it is checking, the same principle as the out-of-band perimeter monitor. Where a monitor genuinely must share a host with what it watches, make it the last thing to be sacrificed: protect it from the out-of-memory killer explicitly and size its limits so the cap itself does not promote it to first victim. Treat the monitoring stack as load-bearing infrastructure, not as a passive observer that is always there.

Why this generalizes. Any monitor co-located with its subject is a latent single point of failure under resource exhaustion, because the failure it exists to report can take it down before it reports. The cheapest fix is independence; the next cheapest is explicit survival priority. Either way, “the monitor is running” is not the same guarantee as “the monitor will still be running when the bad thing happens.”

17 A backup you have never restored is a hope, not a backup

Class of trap. You configure backups, they run on a schedule, the job reports success, and you move on. The failure mode of a backup is silence: it can keep producing nothing, or producing subtly corrupt or incomplete archives, and look perfectly healthy from the outside the entire time. You discover which kind you had only during the disaster, when it is the one moment you cannot

afford to find out. A “the job ran” check confirms the job ran; it confirms nothing about whether what it produced can bring the system back.

The right path. Verify recoverability behaviorally, on a schedule, before you need it. Decrypt a real archive, unpack it, and confirm the contents are intact rather than truncated. Audit completeness against the restore plan, not against what the job happened to capture, which is how I found pieces I assumed were covered and were not. And alarm on the absence of fresh output, so a backup job that quietly stopped is itself a page-able event rather than a surprise.

Why this generalizes. Backups fail toward looking fine, so the only honest signal is to exercise the restore path and to treat silence as failure. This holds for any redundancy whose value is only realized during a rare event: untested failover, an unexercised rollback, a disaster-recovery runbook nobody has walked through. If you have not made it work at least once on purpose, you do not know that it works; you are hoping.

18 A recovery key kept only on the machine it protects is a bootstrap paradox

Class of trap. Your backups are encrypted, which is correct. The key that decrypts them lives on the machine the backups are of, which is the quiet mistake. As long as that machine is healthy you never notice, because you never need to decrypt from anywhere else. The day the machine dies, which is precisely the day you reach for the backups, the only copy of the key you need to read them died with it. You have a vault and no way in.

The right path. Store the recovery key in physically separate escrow, off the machine and off the fleet, and prove the escrowed copy is current rather than assuming it. The proof is concrete: derive the public identity from the escrowed key and confirm it matches the one the live backups were actually encrypted to. An escrow you have not verified against the real backups is a second hope stacked on the first.

Why this generalizes. Any credential whose entire purpose is recovery has to survive the failure it recovers from, which means it cannot share fate with the thing it protects. The general shape is to ask, for every recovery secret, “where is this when the machine is gone,” and to refuse any answer that is “on the machine.”

19 Tear a tunneled service down in dependency order, or you orphan its pieces

Class of trap. Decommissioning a service that reaches the outside through a managed tunnel and an identity gate is more than stopping a container. The pieces have an order. Delete the upstream cloud resource first and the local connector keeps trying to register against something that no longer exists; delete a shared network while a live member is still attached and the removal fails or strands the survivor; rotate or remove a credential that, unknown to you, a different service was also using,

and you break that one instead. The teardown looks done from one angle while it has quietly left orphaned state on another.

The right path. Map every shared resource, networks, credentials, ingress records, before removing anything, and tear down in dependency order: stop the connector and let it deregister cleanly first, confirm the path is inactive, then delete the upstream resources, and only then remove the shared pieces, after migrating any surviving consumer off them. The rule is the same as stopping a service before uninstalling its package: whatever must deregister should deregister before the thing it registered against is gone.

Why this generalizes. This is independent of any particular provider. Any system with a registration relationship and shared dependencies has a correct teardown order, and doing it out of order leaves orphans that resurface weeks later as a name that will not free up or a credential that still works. Removal is a change like any other; it deserves the same ordering discipline as a deploy, and a quick inventory of what else touches each piece before you pull it.

20 Meta-lesson

Every one of these cost real debugging time, and several of the newer ones cost an outage rather than just an afternoon. Almost all of them were invisible until I triggered them; none were findable by reading the official documentation ahead of time. They span the original build, the second hardening pass, the workstation migration, and now the operational-health and recovery work, which tells me the supply of these does not run out as a project matures; it just moves to whatever layer I most recently started trusting. The most recent batch is its own confirmation of that: the moment I trusted the security audit to mean the system was fine, and trusted the backups to mean I could recover, the system found the gap between those assumptions and reality and showed it to me. The way you build a hardened system is by making these discoveries, writing them down, and cross-referencing them across your projects so future-you does not re-discover them at 2 AM on a Tuesday.

I keep the live version of this document in my private planning directory, with the specific tool names and config paths intact because I need them for my own debugging. The public version that you are reading is the generalized one. If any of these patterns saves you an afternoon, that is the whole point of writing them down.